Basic Arithmetic

> We'll do it tomorrow
>
> — TODO

We'll start by giving a formal account of the simplest practical language – that of *first-order* arithmetic expressions, like $x + y$ and $a + (5 \cdot b) + \sin(3)$.

Rather than complicate our language by introducing binary operators (like $x + y$), unary operators (like $-x$), and n-ary function applications (like $f(x, y, z)$), we can instead normalize everything to *S-expressions* – or *sexprs* – of the form $(f\ e_1\ \ldots\ e_n)$. We can then uniformly write:

- binary operations like $x + y$ as binary sexprs $(+\ x\ y)$,
- unary operations $-x$ as unary sexprs $(-\ x)$,
- function appplications $f(x, y, z)$ as n-ary sexprs $(f\ x\ y\ z)$.

In particular, the *operator* f always comes first, and there is no order-of-operations – all bracketing is explicit. Restricting ourselves to *first-order* sexprs means that we always require operators f to be atomic symbols drawn from a fixed set O – in particular, we don't support *partial application*, like $((\mathsf{add}\ 2)\ 3)$, since that would require $f = (\mathsf{add}\ 2)$ to be a compound expression.

Formally, we define our untyped *syntax* as follows:

Definition 0.1 (First-order S-expressions): Fixing
- a set of *variables* $x \in N$
- a set of *operations* $f \in O$
- a set of *constants* $c \in V$

we define the set of *first-order S-expressions* $e \in \mathsf{STm}_N(O, V)$ to be given by the grammar

$$e ::= x \mid c \mid (f\ e^*)$$

We can define a typing relation over this syntax, parametrized by a set of types $A \in \mathcal{T}$, in the usual manner. We begin by fixing some conventions:

Definition 0.2 (Typing Context): Given a set of variables N and a set of types $\mathcal{T}$, we define the set of *contexts* $\Gamma : \mathsf{Ctx}_N(\mathcal{T})$ to be the set of lists of bindings $x : A$, given by the grammar

$$\Gamma ::= \cdot \mid \Gamma, x : A \text{ where } x \notin \mathsf{def}(\Gamma)$$

Here, $\mathsf{def}(\Gamma)$ denotes the set of *defined variables* in Γ, defined as

$$\mathsf{def}(\cdot = \emptyset) \qquad \mathsf{def}(\Gamma, x : A) = \mathsf{def}(\Gamma) \cup \{x\}$$

Given a context Γ, we can hence define a partial function from variables N to types $\mathcal{T}$ as follows:

$$(\mathsf{pty}(\Gamma), x : A)(x) \coloneqq A \qquad \mathsf{pty}(\Gamma, y : A)(x) \coloneqq \mathsf{pty}(\Gamma)(x) \text{ where } x \in \mathsf{def}(\Gamma)$$

It follows that we can view a context Γ as isomorphic to:

- a *finitely supported partial function* $\mathsf{pty}(\Gamma) : N \rightharpoonup \mathcal{T}$ – where the support of Γ is the set of defined variables $\mathsf{def}(\Gamma)$
- a *total order* on the set of defined variables $\mathsf{def}(\Gamma)$ – given by the order in which they appear in the context

In general, we'll define $\Gamma(x) := \mathsf{pty}(\Gamma)(x)$.

Definition 0.3 (Typed Constants): We define a set of *typed constants* $c \in V$ to be a set V equipped with a *typing relation* $V \nrightarrow \mathcal{T}$ – written $c : A$ to mean "c has type A".

Note that constants are typed with a *relation* – that is, the same constant c can be assigned multiple types A, B, C.

Definition 0.4 (Typed Operations): We define a set of *typed operations* $f \in O$ to be a set O equipped with a *typing relation* $O \times \mathcal{T}^* \nrightarrow \mathcal{T}$ – written $f : [A_1, ..., A_m] \to B$ to mean "f takes inputs $[A_1, ..., A_m]$ to output B".

We've now got what we need to define a well-typed first-order S-expressions:

Definition 0.5 (Typed First-order S-expressions): Fixing

- a set of *variables* $x \in N$
- a set of *typed operations* $f \in O$ equipped with a *typing relation* $O \times \mathcal{T}^* \nrightarrow \mathcal{T}$ – where "f takes inputs $[A_1, ..., A_m]$ to output B" is written $f : [A_1, ..., A_m] \to B$
- a set of *typed constants* $c \in V$ equipped with a *typing relation* $V \nrightarrow \mathcal{T}$ – where "c has type A" is written $c : A$

we define a typing relation on first-order S-expressions e via the following rules:

- **TODO 1: Var**
- **TODO 2: Const**
- **TODO 3: Op**

TODO 4: Subtyping, weakening and substitution

TODO 5: Define the denotational semantics

- **Pure**
- **Monadic**
- **Categorical**